

Technical Documentation

Generated on: 2026-03-26T19:01:33.981Z

README.md

Documentation Display (docs_display)

Purpose: Consolidated, developer-focused technical documentation for the product. Use these templates as the authoritative technical source for developers, QA, DevOps, security, and architects.

Structure:

- 1_product_requirements.md
- 2_architecture.md
- 3_api_documentation.md
- 4_data_database.md
- 5_security_compliance.md
- 6_devops_deployment.md
- 7_testing_qa.md
- 8_release_versioning.md
- 9_monitoring_observability.md
- 10_technical_roadmap.md
- code_quality_and_reports.md
- build_and_deployment_reports.md
- openapi.yaml
- diagrams/sequence_diagrams.mmd
- diagrams/er_diagram.mmd

How to use:

- Fill in TODO sections with concrete system values pulled from the codebase or engineering leads.
- Keep `openapi.yaml` updated as the canonical API contract and generate API clients from it.
- Store finalized diagrams as both `.mmd` and exported images in `docs_display/diagrams/`.

Owner: Product / Engineering

1_product_requirements.md

1. Product Technical Requirements

Functional Specifications (Tech-Focused)

- Overview: TODO — brief feature list derived from codebase and product backlog.
- For each feature:
 - Purpose: TODO
 - Inputs: HTTP params, events, file formats
 - Outputs: responses, events, side effects
 - Edge cases: validation, retries, failure modes
 - Sequence diagram: reference [diagrams/sequence_diagrams.mmd](#)

Example feature template

- Name: "Create Booking"
- Endpoint: `POST /api/v1/bookings`
- Inputs: `user_id` (uuid), `slot_id` (uuid), `metadata` (object)
- Success: 201 Created + booking object
- Errors: 400 (validation), 401 (auth), 409 (conflict)

Non-Functional Requirements

- Performance: P95 latency $\leq$ TODO ms for core endpoints
- Scalability: horizontal autoscaling for API and workers
- Security: TLS everywhere, role-based access
- Compliance: GDPR data residency & right-to-delete
- Reliability: target uptime 99.9% (SLA)

Acceptance Criteria

- Each feature must have unit + integration tests covering happy + error flows.
- API contract published in `openapi.yaml`.

2_architecture.md

2. Architecture & Design Documentation

System Overview

- High-level components: API gateway, auth service, application services, background workers, database, cache, object storage, analytics.
- Deployment: containers (Docker) in cloud provider / Kubernetes.

Component / Module Design

- For each module list: responsibilities, interfaces, events produced/consumed, failure modes.

Example: Booking Service

- Responsibilities: create/modify/cancel bookings, enforce business rules.
- Interfaces: `POST /bookings`, internal event `booking.created`.
- Data: booking table (FKs to user, slot).

Sequence Diagrams

- See `diagrams/sequence_diagrams.mmd` for mermaid sequence diagrams.

Data Flow Diagrams

- Describe data ingress (API -> auth -> service -> DB), enrichment, analytics export.

Service Interactions

- Synchronous: REST/HTTP for queries and commands.
- Asynchronous: message queue (e.g., Redis/Stream/Kafka) for events and background jobs.

Design Decisions

- Rationale for microservice vs monolith, data ownership boundaries, eventual consistency tradeoffs.

TODOs:

- Populate component responsibilities and diagrams from the codebase.

sequence_diagrams.mmd

%% Mermaid sequence diagram suggestions
sequenceDiagram participant Client participant APIGW as API\ Gateway participant Auth as Auth\ Service participant Book as Booking\ Service participant DB as Database

```
Client->>APIGW: POST /api/v1/bookings
APIGW->>Auth: Validate token
Auth-->>APIGW: 200 OK
APIGW->>Book: CreateBooking(command)
Book->>DB: INSERT booking
DB-->>Book: 201 Created
Book-->>APIGW: BookingCreated
APIGW-->>Client: 201 Created + body
```

dfd.mmd

%% Data Flow Diagram (Level 0) - Mermaid flowchart
flowchart LR
 Client[Client App] -->|HTTP Requests| API[API Gateway]
 API --> Auth[Auth Service]
 API --> BookingService[Booking Service]
 BookingService --> DB[(Postgres DB)]
 BookingService --> MQ[(Message Queue)]
 MQ --> Worker[Background Worker]
 Worker --> Storage[(Object Storage)]
 Analytics[Analytics] <-- DB
 API --> Cache[(Redis Cache)]

uml_class.mmd

%% UML class diagram (mermaid)
classDiagram
 class User {
 +UUID id
 +String name
 +String email
 +createBooking()
 }
 class Booking {
 +UUID id
 +UUID user_id
 +UUID slot_id
 +String status
 +confirm()
 +cancel()
 }
 class Slot {
 +UUID id
 +DateTime start
 +DateTime end
 }

```
User "1" -- "*" Booking : places
Slot "1" -- "*" Booking : contains
```

flowchart.mmd

%% Flowchart for booking process
flowchart TD
 Start([Start]) --> AuthCheck{Authenticated?}
 AuthCheck -- No --> Login[Redirect to Login]
 AuthCheck -- Yes --> SelectSlot[Select Slot]
 SelectSlot --> Confirm[Confirm Booking]
 Confirm --> Payment{Payment Required?}
 Payment -- Yes --> Pay[Process Payment]
 Pay --> CreateBooking[Create Booking]
 Payment -- No --> CreateBooking
 CreateBooking --> Notify[Send Confirmation]
 Notify --> End([End])

bpmn.mmd

%% BPMN-like process (represented with flowchart)
flowchart LR
 User((User)) -->|Request| APIGateway[API Gateway]
 APIGateway --> AuthService[Auth Service]
 AuthService -->|valid| BookService[Booking Service]
 BookService --> CheckAvailability{Available?}
 CheckAvailability -- Yes --> Reserve[Reserve Slot]
 Reserve --> PaymentGateway[Payment Gateway]
 PaymentGateway -->|success| Confirm[Confirm Booking]
 Confirm --> Notify[Notify User]
 CheckAvailability -- No --> Reject[Reject Request]

3. API Documentation

Overview

- Canonical API contract: `openapi.yaml` (update as implementation changes).
- Use semantic versioning for APIs: `/api/v1/...`, `/api/v2/...`

Authentication

- OAuth2 / JWT bearer tokens (or API keys for machine-to-machine). Include scopes and role mappings.

Rate Limits

- Default: TODO requests/min per API key.
- Burst: TODO.

Error Model

- Standard JSON error: `{ "code": "ERR_CODE", "message": "human message", "details": {...} }`
- HTTP mapping: 4xx client errors, 5xx server errors. List of common codes:
 - 400: Bad Request (validation failed)
 - 401: Unauthorized
 - 403: Forbidden
 - 404: Not Found
 - 409: Conflict
 - 429: Too Many Requests
 - 500: Internal Server Error

Endpoints (examples)

POST `/api/v1/bookings`

- Request:

```
{
  "user_id": "uuid",
  "slot_id": "uuid",
  "metadata": { }
}
```

- Response 201:

```
{
  "id": "uuid",
  "status": "confirmed",
  "created_at": "ISO8601"
}
```

Versioning and Deprecation

- Major version for breaking changes. Maintain compatibility for two versions where feasible.

- Deprecation policy: announce 90 days, provide migration guide.

Tools & Artifacts

- `openapi.yaml` — machine-readable spec.
- Postman collection / generated clients — keep alongside API Cl.

openapi.yaml

```
openapi: 3.0.3
info:
  title: Product API
  version: 1.0.0
  description: Canonical API contract — fill endpoints and schemas.
paths:
  /api/v1/bookings:
    post:
      summary: Create a booking
      tags: [Bookings]
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CreateBooking'
      responses:
        '201':
          description: Created
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Booking'
        '400':
          $ref: '#/components/responses/BadRequest'
components:
  schemas:
    CreateBooking:
      type: object
      required: [user_id, slot_id]
      properties:
        user_id:
          type: string
        slot_id:
          type: string
        metadata:
          type: object
    Booking:
      type: object
      properties:
        id:
          type: string
        status:
          type: string
        created_at:
          type: string
          format: date-time
  responses:
    BadRequest:
      description: Validation failed
      content:
        application/json:
          schema:
            type: object
            properties:
              code:
                type: string
              message:
                type: string
```

4. Data & Database Documentation

Database Schema (overview)

- Primary stores: relational database (Postgres) — tables: users, bookings, slots, payments, audit_logs.
- Caching: Redis for session/counters.

ER Diagram

- See `diagrams/er_diagram.mmd` for a mermaid ER starter.

Data Dictionary (example)

- bookings:
 - `id` (uuid): PK
 - `user_id` (uuid): FK -> users.id
 - `slot_id` (uuid): FK -> slots.id
 - `status` (varchar): enum(created, confirmed, cancelled)
 - `created_at` (timestamp)

Relationships

- One `user` to many `bookings`.

Indexes & Performance

- Indexes: `bookings(user_id)`, `bookings(slot_id)`, `bookings(created_at)` — tune for read patterns.

Data Lifecycle & Retention

- Raw retention: 90 days in primary DB, archival to object storage for 7 years (configurable).
- Deletion: soft-delete + GDPR purge endpoint that removes PII within configured SLA.

Backups & Recovery

- Daily DB snapshots, WAL archiving for point-in-time recovery.

5_security_compliance.md

5. Security & Compliance Documentation

Authentication / Authorization

- Auth flow: OAuth2 / JWT issued by Auth Service. Token refresh, scopes, and RBAC mapping.

Encryption Standards

- In-transit: TLS 1.2+ with strong ciphers.
- At-rest: AES-256 for DB backups and object storage.

Key Management

- Keys stored in KMS (cloud provider) or Vault; rotation policy: 90 days for symmetric keys, yearly for asymmetric.

Threat Model & Mitigations

- Top threats: injection, auth bypass, data exfiltration.
- Mitigations: input validation, parameterized queries, WAF, least privilege.

Vulnerability Management

- Regular SCA and dependency scans; CVE triage within 48 hours for critical.

Audit & Logging

- Immutable audit logs for sensitive actions; retention policy per compliance.

Compliance Controls

- GDPR: data subject requests, deletion flows, data portability.

6_devops_deployment.md

6. DevOps / Deployment Documentation

Environments

- Development: local docker-compose
- Staging: mirrors prod, feature branches deployed by CI
- Production: Kubernetes cluster with autoscaling

CI/CD Pipeline

- Steps:
 1. Lint & static analysis
 2. Unit tests
 3. Build artifacts (containers)
 4. Integration tests in ephemeral environment
 5. Deploy to staging
 6. Promote to production via deployment pipeline

Build & Deployment Instructions

- Build container image:

```
docker build -t registry.example.com/product/service:TAG .
```

- Deploy: CI handles image push; use helm charts under `infra/helm` (TODO: path)

Rollback Procedures

- Use previous image tag and trigger deployment rollback via Helm or k8s rollout undo.

Infrastructure as Code

- Store Terraform modules in `infra/terraform/` with state in remote backend.

Environment Configuration

- Config via environment variables and secrets manager — list required variables here.

7_testing_qa.md

7. Testing & QA Documentation

Test Types

- Unit Tests: fast, isolated, run on each PR.
- Integration Tests: cover interactions between services and DB.
- End-to-End: scenario tests through API/UI flows.
- Performance: load and stress tests with targets.
- Regression: nightly runs for critical flows.

Test Coverage & Reporting

- Tooling: coverage reports (e.g., `nyc`, `coverage.py`). Minimum acceptable coverage: TODO%.

Test Data & Environments

- Use deterministic fixtures; anonymized production snapshots for staging when needed.

Security Testing

- SAST during CI; periodic DAST and pentests.

Test Run Examples

```
npm run test:unit
npm run test:integration
```

8_release_versioning.md

8. Release & Versioning Documentation

Versioning Strategy

- Semantic Versioning: MAJOR.MINOR.PATCH.

Release Process

- Create release candidate branch, run full CI, smoke tests, then tag and publish.

Changelog

- Maintain changelog.md or rely on Git-based generation (conventional commits).

Migration & Backward Compatibility

- Document DB migrations and data migrations in `infra/migrations` and migration guides.

9_monitoring_observability.md

9. Monitoring & Observability

Metrics

- Key metrics: request latency (p50/p95/p99), throughput (requests/sec), error rate, CPU/memory.

Logging

- Structured JSON logs with fields: timestamp, service, level, request_id, user_id (if applicable), message.

Tracing

- Distributed tracing with W3C Trace Context; sample spans for user requests and background jobs.

Alerts & Dashboards

- Threshold-based alerts for error rate spike, high latency, resource exhaustion.
- Dashboards: API health, consumer experience, system health.

Incident Response

- Playbook: detect -> triage -> mitigate -> root cause -> postmortem.

10_technical_roadmap.md

10. Technical Roadmap & Decisions

Current State

- Summarize current infra, tech stack, and outstanding technical debt.

Planned Work

- Short-term (0-3 months): TODO
- Mid-term (3-12 months): TODO
- Long-term (12+ months): TODO

Architectural Decisions

- Record ADRs (Architecture Decision Records) for major choices and rationale.

Known Technical Debt

- List and prioritize debt items with owners and remediation plans.

code_quality_and_reports.md

Code Quality & Reports

Static Analysis & Linting

- Tools: ESLint / flake8 / SonarQube. Configure rules and thresholds in CI.

Technical Debt & Refactoring Logs

- Track debt tickets and link to PRs addressing them.

Quality Metrics

- Lint violations per PR, test pass rates, cyclomatic complexity baselines.

build_and_deployment_reports.md

Build & Deployment Reports

CI/CD Health

- Pipeline success rate: TODO%
- Average build time: TODO minutes

Build Artifacts

- Registry: registry.example.com/product/

Deployment Frequency & Trends

- Frequency of deploys to staging/production, mean time to recovery (MTTR).